Report

Assignment 2

March 12, 2025

Student:
Zoë Azra Blei - 2695271
Eleni Liarou - 2868489

Lecturer:
Oliver Champion

Course:
Deep Learning for Medical Imaging

Course code:
XM_0151

1 Exercise 1 - Calculate the trainable parameters

C1: 156 parameters The paper mentions that we have a total of 6 feature maps, each with a receptive field of size of 5x5 and bias of 1. So each unit in a feature map has 25 trainable coefficients(weights) and 1 bias.

A general formula for calculating the parameters for a filter is the following:

$$\text{parameters per filter} = (\text{filter height} \times \text{filter width} \times \text{input channels}) + \text{bias} \tag{1}$$

This result in: parameters per filter $= (5 \cdot 5 \cdot 1) + 1 = 26$

As there are six filters, the total number of parameters is: 26 x 6 = 156

S2: 12 parameters In this case we are not learning spatial relationships, but rather downsampling the data and learning a scale with which we can reduce the size of the image. Per kernel we have 1 weight and 1 bias. As there are 6 filters the total number of parameters is $2 \cdot 6 = 12$.

C3: 1516 parameters The paper mentions the following regarding the organization of the C3 layer:

1. The first 6 C3 feature maps take input from every contiguous subsets of 3 S2 maps

2. The next 6 C3 feature maps take input from every contiguous subsets of 4 S2 maps

3. The next 3 feature maps take input from some discontinuous subsets of 4 S2 maps

4. The last feature map takes input from all of 6 S2 maps

Using equation 1 from above, we can calculate the number of parameters for each instance:

params instance 1 : $(5 \cdot 5 \cdot 3) + 1 = 76$

As there are 6 C3 feature maps: $6 \cdot 76 = 456$

params instance 2 : $(5 \cdot 5 \cdot 4) + 1 = 101$

As there are 6 C3 feature maps: $6 \cdot 101 = 606$

params instance 3 : $(5 \cdot 5 \cdot 4) + 1 = 101$

As there are 3 C3 feature maps: $3 \cdot 101 = 303$

params instance $4 = (5 \cdot 5 \cdot 6) + 1 = 151$

As there is 1 C3 feature map: $1 \cdot 151 = 151$

To calculate the total number of parameters for the C3 layer, we no add the parameters of the different instances together: Total number of parameters: 456 + 606 + 303 + 151 = 1516

S4: 32 parameters Layer 4 is sub-sampling layer with 16 feature maps of size 5x5, each unit in each feature map is connected to 2x2 neighborhoud in the corresponding feature map in C3, in a similar way as C1 and S2. Each feature map has one trainable coefficient and one trainable bias per future map tehrefore the total number of trainable parameters is $16 \cdot 2 = 32$.

C5: 48,120 parameters

Based on the paper layer 5 has 120 feature maps, each unit in C5 is connected to a 5x5 neighborhood in each of the 16 feature maps of S4. We calculate the number of parameters using equation 1 again: Each unit in C5 has $5 \cdot 5 = 25$ inputs per feature map in S4. Since there are 120 feature maps in C5, there are 120 units in total. Each unit in C5 is connected to every unit in the receptive field on all 16 S4 feature maps, so each unit has $16 \cdot 25 = 400$ weights (25 weights per feature map, and 16 feature maps in S4). Each unit in C5 has one bias term, and since there are 120 feature maps, there are 120 bias terms in total.

So the total number of trainable parameters is:

$120 \cdot (400 + 1) = 48,120$

F6: 10,164

Input of F6 comes from output of C5, which has 120 feature maps. Each feature map has a size of 1x1, so there are 120 units in total. F6 has 84 feature maps. Each unit in F6 is fully connected to every unit i=n C5, so the number of weights per unit is 120 (one for each unit from C5). Therefore, the total number of weights for the entire layer is 120 x 84. Each unit in F6 has one bias, so there are 84 biases in total.

Therefore, the total number of trainable parameters is: $84 \cdot 120 + 84 = 10,164$

2 Exercise 2 - U-Net

The formula for calculating output dimensions after convolution(no padding and stride s) is:

$$\text{Output size} = \left\lfloor \frac{\text{Input size} - \text{Kernel size}}{\text{Stride}} \right\rfloor + 1$$

.

Convolution formula with padding p:

$$\text{Output size} = \left\lfloor \frac{\text{Input size} + 2p - \text{Kernel size}}{\text{Stride}} \right\rfloor + 1$$

2.1 A

- Dimensions for A A (3x3x1) convolution is implemented with stride (2,2,1), zero padding and 64 output features.

  For each dimension:

$$A_x = \left\lfloor \frac{69-3}{2} \right\rfloor + 1 = \left\lfloor \frac{66}{2} \right\rfloor + 1 = 34$$

$$A_y = \left\lfloor \frac{69-3}{2} \right\rfloor + 1 = 34$$

$$A_z = \left\lfloor \frac{34-1}{1} \right\rfloor + 1 = 34$$

  Thus, $A = (34 \times 34 \times 34)$ with 64 feature maps.

- Dimensions for B A 2×2×2 max pooling, with stride $2 \times 2 \times 2$ is implemented.

$$B_x = \left\lfloor \frac{34}{2} \right\rfloor = 17$$

$$B_y = \left\lfloor \frac{34}{2} \right\rfloor = 17$$

$$B_z = \left\lfloor \frac{34}{2} \right\rfloor = 17$$

  Thus, $B = (17 \times 17 \times 17)$ with 64 feature maps.

- Dimensions for C A 3×3×3 convolution, with stride $1 \times 1 \times 1$, padding 1, and 128 feature maps is implemented.

  Since padding is 1, and stride is 1:

$$C_x = \left\lfloor \frac{17 + 2(1) - 3}{1} \right\rfloor + 1 = 17$$

$$C_y = 17$$

$$C_z = 17$$

Thus, $C = (17 \times 17 \times 17)$ with 128 feature maps.

- Dimensions for D

  The max pooling operation uses a $(2 \times 2 \times 2)$ filter with a stride of 2. However, 17 is an odd number, meaning the pooling operation will not evenly divide the input. This means that D's dimensions are not well-defined because we cannot evenly apply max pooling to $(17 \times 17 \times 17)$

2.2 B

Each max pooling operation reduces the spatial dimensions by a factor of 3. To ensure a valid reconstruction in the up-sampling path, the input size must be divisible by 3^n, where n is the number of down-sampling layers. From the given diagram, we see there are 3 down-sampling steps. Therefore the input must be divisible by $3^3 = 27$. Therefore, the smallest possible square input image would be 27×27 pixels.

3 Exercise 3 - CNN

In this exercise, we implemented two deep learning models for the ISIC 2019 skin lesion classification challenge: a custom convolutional neural network (CustomConvNet) and a transfer learning model based on ResNet50 (TransConvNet). The goal was to classify dermoscopic images with a primary focus on recall and F1 score.

Methodology

Custom convolutional network: The improved SimpleConvNet boosts feature extraction and generalization by adding depth, a residual connection, and a refined classifier. The updated model expands to five convolutional layers (32→512 filters), capturing more complex lesion patterns. A skip connection enhances gradient flow, preventing vanishing gradients and improving training. LeakyReLU replaces ReLU to prevent dead neurons, ensuring continuous gradient updates. The classifier now includes global average pooling, dense layers (512→256→128→1), and dropout to reduce overfitting.

Transfer learning convolutional network: We used a ResNet50 model pre-trained on ImageNet, replacing its fully connected layer with a custom classifier. To balance feature extraction and computational efficiency, early layers were frozen, while the last residual block was fine-tuned.

Data Augmentation: The code applies random horizontal/vertical flips and Gaussian blur for image augmentation. This enhances generalization by making the model more robust to varying orientations and noise, teaching it to recognize patterns regardless of orientation, and simulating different levels of image sharpness. The results of these augmentations can be seen in Figure 1.

Focal Tversky Loss: To prioritize recall, we implemented the Focal Tversky Loss, which penalizes false negatives more heavily. The Tversky index balances false positives and false negatives, with an $\alpha(=0.7)$ parameter controlling the trade-off. To emphasize hard-to-classify cases, a focal term $\gamma(=0.75)$ is applied, down-weighting easy examples. This loss function is particularly suited for biomedical data, where minimizing false negatives is critical for accurate lesion detection.

Evaluation metrics: Given the critical nature of cancer detection, we prioritized recall over precision when assessing the model, as false negatives (missed cancer cases) are far more detrimental than false positives. Additionally, we emphasize validation scores over training scores, as validation provides a more accurate reflection of the model's ability to generalize to unseen data, ensuring its effectiveness in real-world applications.

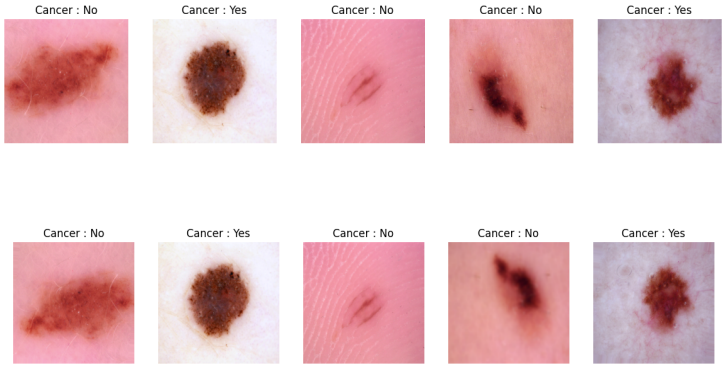

Figure 1: Data before(first row) and after(second row) augmentation

Network	Learning Rate	Epochs	Batch Size	Optimizer	Loss
CustomConvNet	0.0001	50	64	Adam	Focal Tversky
TransConvNet	0.0001	50	64	Adam	Focal Tversky
SimpleConvNet	0.01	10	16	SGD	BCE
UNet	0.0001	50	32	Adam	BCE

Table 1: Hyperparameters for SimpleConvNet, CustomConvNet, TransConvNet and UNet

Results and Interpretation

Figure 2 presents the model performance metrics. Both TransConvNet and CustomConvNet achieved similar recall and F1 scores, demonstrating their effectiveness in minimizing false negatives. However, TransConvNet outperformed CustomConvNet in accuracy and precision, likely due to the advantages of transfer learning and pre-trained feature extraction.

CustomConvNet showed significant improvements over SimpleConvNet, benefiting from increased depth, skip connections, and Focal Tversky Loss, which enhanced recall. Additionally, data

augmentation applied to both CustomConvNet and TransConvNet helped improve generalization by making the models more robust to variations in dermoscopic images. In contrast, SimpleConvNet exhibited unstable recall and higher loss, highlighting its limitations in complex feature extraction.

The use of Focal Tversky Loss in CustomConvNet and TransConvNet contributed to improved recall by prioritizing hard-to-classify cases. Overall, TransConvNet's higher accuracy and f1 score make it the preferred model for balancing sensitivity and specificity in cancer detection.

Figure 2: Performance Metrics for CNN

4 Exercise 4 - U-NET

In this exercise, we implemented the U-NET network based on the image segmentation principles introduced in the work of Ronneberger, et. al. [1]. The aim was to optimize the network and training strategies by changing the network structure, tuning training parameters, and adapting the data augmentation.

Methodology

The model is based on the U-Net architecture proposed by Ronneberger et al.[1], with several modifications. Padding was added to all convolutional layers to preserve spatial dimensions throughout the network. The encoder follows a hierarchical structure, where each stage consists of two convolutional layers followed by max pooling for downsampling. The decoder uses transposed convolutions to restore spatial resolution, incorporating skip connections to retain spatial information. Unlike the original U-Net, skip connections include learnable 1×1 convolutional layers to refine encoder features before merging with the decoder. LeakyReLU is used as the activation function instead of

ReLU to mitigate the issue of dead neurons, and batch normalization is applied to improve training stability. A dropout layer is introduced in the bottleneck to enhance regularization and prevent overfitting.

To further optimize the network the following strategies were incorporated:

Data augmentation: Same transformations as before were applied.

Loss function: We experimented with different loss functions, including a combination of Binary Cross-Entropy (BCE) with Dice loss and Focal Tversky loss. BCE loss alone yielded the best performance, balancing segmentation accuracy and training stability.

Evaluation Metrics: Same as for CNN.

Results and Interpretation

The performance metrics for the U-Net model, as shown in Figure 3, indicate stable and consistent improvement throughout training. The validation recall fluctuates but demonstrates an overall increasing trend, suggesting that the model is progressively enhancing its ability to detect positive cases. This variability may be indicative of sensitivity to class imbalance or challenging samples. In terms of validation precision, the curve rises rapidly at the beginning and then stabilizes. Validation accuracy exhibits a steady upward trend, ultimately stabilizing at a high value of approximately 0.94, reflecting that the model is making correct predictions for most pixels. The validation F1-score follows a similar pattern to recall, improving early in training and stabilizing around 0.85, signifying a good balance between precision and recall. Lastly, the validation loss consistently decreases, confirming that the model is learning effectively without experiencing significant overfitting.

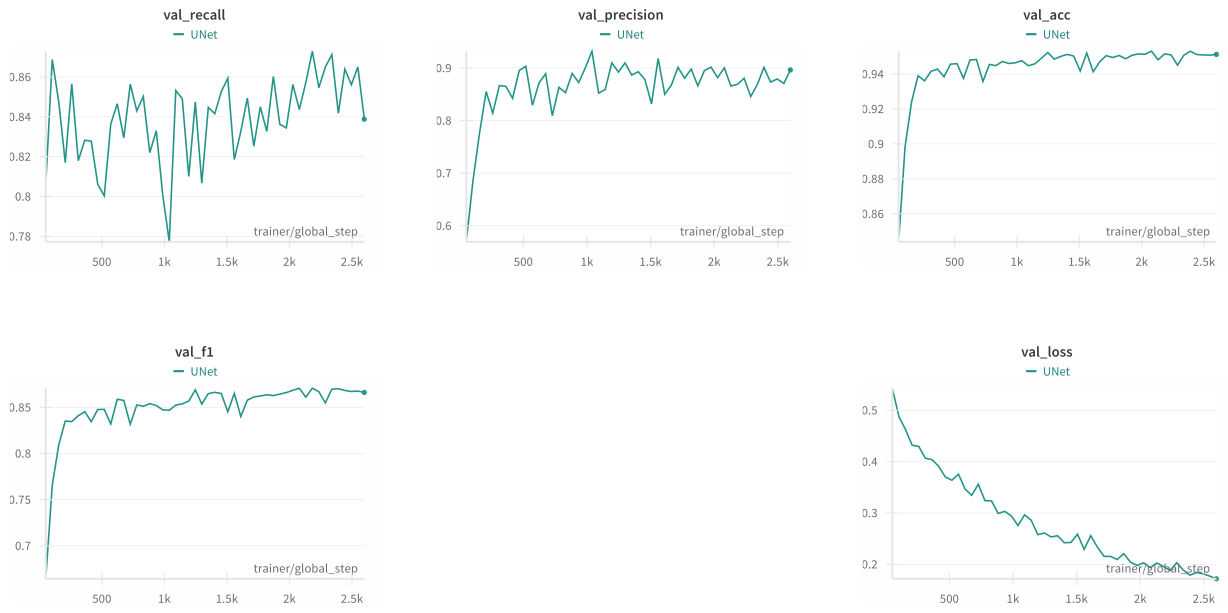

Figure 3: Performance Metrics for UNet

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," May 2015. arXiv:1505.04597 [cs].